

In [267... `import gymnasium as gym`
`from typing import Optional`

We've implemented a gym environment for playing blackjack properly... will do models later. What sort of models are needed to effectively play blackjack given only symbolic knowledge of the game? We have set up a symbolic only environment for the agent to play with, and will see what sort of models we can use to train in this gym.

```
In [352... class BlackjackTable(gym.Env):

    def __init__(self):
        import numpy as np
        self.observation_space = gym.spaces.Dict(
            {
                "bet": gym.spaces.Discrete(10**6),
                "bankroll": gym.spaces.Discrete(10**6),
                #i'm actually kinda unsure if we need a bankroll to begin with...
                "playercards": gym.spaces.Sequence(gym.spaces.Discrete(52)),
                "dealercards": gym.spaces.Sequence(gym.spaces.Discrete(52))
            }
        )
        self.action_space = gym.spaces.OneOf((gym.spaces.Box(low = 1, high = 1000,
#observe that the action space spits out a tuple. If the first element is 0
# if the first is a 1, its a hit/stand, where stand is 0, hit is 1.
            self.deck = np.arange(52) #the deck begins as a sequence of cards
            self.bet = 0 #how much has been bet on the current hand.
            self.bankroll = 0 #this is how much money the player has
            self.position = 0 #this is how far along in the deck the table is. The pos
            self.playercards = () #this is a sequence of cards for the player
            self.dealercards = () #this is a sequence of cards for the dealer. Ultimate

    def reset(self, seed: Optional[int] = None, options: Optional[dict] = None):
        super().reset(seed=seed)
        self.np_random.shuffle(self.deck)
        self.bankroll = 100
        self.bet = 0 #reset the bet
        self.position = 0 #start at the top
        self.playercards = () #player no cards
        self.dealercards = () #this is a sequence of cards for the dealer. Ultimate

    def _get_obs(self):
        return {"bankroll": self.bankroll,
                "bet": self.bet,
                "playercards": self.playercards,
                "dealercards": self.dealercards}

    def _get_info(self):
        return {"pastcards": self.deck[:self.position]}

    def _reset_board(self, reward): #for resetting the board. gives appropriate rew
        self.playercards = ()
        self.dealercards = ()
        self.bankroll += reward
        self.bet = 0 #reset bet
```

```

    return reward

def step(self, action):
    import numpy as np
    if ((not self.bet) and (52 - self.position) < 12) or self.bankroll == 0:
        #end game, since there is no bet and we have limited cards...or no mone
        return self._get_obs(), self.bankroll, True, False, self._get_info()
    if ((not self.bet) and action[0]) or (not action[0] and self.bet) or (not a
        #if the user submits a non-betting action with no cards we do nothing.
        #similar if betting action submitted with cards
        #and if bet is zero...
        return self._get_obs(), 0, False, False, self._get_info()
    if not (self.bet or action[0]): #we've got no cards and they bet properly
        self.playercards = tuple(self.deck[self.position: self.position + 2]) #
        self.dealercards = tuple([self.deck[self.position + 2]]) #give the deal
        self.position += 3
        self.bet = action[1]
        self.bankroll -= self.bet
        reward = 0
        if np.max(BlackjackTable.getValue(self.playercards)) == 21: #if blackja
            self.position += 1 #imagine we gave the dealer the last card.
            reward = self._reset_board(np.ceil(self.bet * 2.5))
        return self._get_obs(), reward, False, False, self._get_info()
    if self.bet and action[0]: #we've got a bet and a choice!
        if action[1]: # if hitting...
            self.playercards += (self.deck[self.position],)
            self.position += 1
            reward = 0
            if np.min(BlackjackTable.getValue(self.playercards)) > 21: #bust on
                reward = self._reset_board(0)
        else: #if standing
            while np.min(BlackjackTable.getValue(self.dealercards)) < 17: #Loop
                self.dealercards += (self.deck[self.position],)
                self.position += 1
            if np.min(BlackjackTable.getValue(self.dealercards)) > 21: #dealer
                reward = self._reset_board(2 * self.bet)
            else: # we need to compare the values...
                pv = BlackjackTable.getValue(self.playercards)
                dv = BlackjackTable.getValue(self.dealercards)
                pv = pv[-1] if pv[-1] <= 21 else pv[0]
                dv = dv[-1] if dv[-1] <= 21 else dv[0]
                if pv == dv:
                    self._reset_board(self.bet) #push
                elif pv > dv:
                    reward = self._reset_board(2 * self.bet) #player win
                else:
                    reward = self._reset_board(0)
    return self._get_obs(), reward, False, False, self._get_info()

def getValue(hand):
    import numpy as np
    handval = np.mod(hand, 13)
    handval[handval == 0] = 10
    handval[handval > 10] = 10
    v = np.sum(handval)
    if np.any(handval == 1):

```

```

    return [v, v + 10]
    # comments comments comments
    return [v]

```

```

In [353... def readoutput(output):
    import numpy as np
    print("Bankroll is:", output[0]["bankroll"])
    print("Current bet is:", output[0]["bet"])
    print("Player has:", BlackjackTable.getValue(output[0]["playercards"]))
    print("Dealer has:", BlackjackTable.getValue(output[0]["dealercards"]))
    handval = np.mod(output[4]["pastcards"], 13)
    handval[handval == 0] = 13
    print("Previous cards:", handval)
    handval[handval == 1] = 14
    print("Count is", np.sum(handval < 7) - np.sum(handval > 8))
    print("Game Over" if output[2] else "Continue")

```

```

In [354... tenv = BlackjackTable()
tenv.reset()

```

```

In [355... hand = tenv.observation_space.sample()['playercards']
print("Hand:", hand)
print("Value:", BlackjackTable.getValue(hand))

```

```

Hand: (np.int64(38), np.int64(48))
Value: [np.int64(19)]

```

```

In [356... tenv.reset()
tenv._get_obs()

```

```

Out[356... {'bankroll': 100, 'bet': 0, 'playercards': (), 'dealercards': ()}

```

```

In [357... readoutput(tenv.step((0,1)))#betting

```

```

Bankroll is: 99
Current bet is: 1
Player has: [np.int64(10)]
Dealer has: [np.int64(3)]
Previous cards: [6 4 3]
Count is 3
Continue

```

```

In [358... readoutput(tenv.step((1, 0)))#playing

```

```

Bankroll is: 99
Current bet is: 0
Player has: [np.float64(0.0)]
Dealer has: [np.float64(0.0)]
Previous cards: [ 6  4  3  2 10  4]
Count is 4
Continue

```

```

In [ ]:

```